

Builder Brain — Complete Architecture, Flowcharts & Master Todo Documentation

1. Project Vision

Builder Brain is an AI-powered autonomous software engineering system designed to:

- Understand user requirements
- Ask contextual questions
- Plan applications automatically
- Generate frontend, backend, and database code
- Execute applications inside containers/sandboxes
- Manage multi-agent orchestration
- Process uploaded documents
- Maintain memories and contextual understanding
- Generate documentation automatically
- Support future framework and database expansion

The platform is being designed as a production-grade AI software engineering ecosystem.

2. Current Tech Stack

Frontend

- HTML
- CSS
- JavaScript
- React
- Angular
- Next.js
- Tailwind CSS
- Vanilla CSS

Backend

- Node.js
- Express.js

Database

- Supabase

Architecture Pattern

- MVC Architecture

Runtime Layer

- WebContainer (Integrated)
 - Sandbox (Pending)
-

3. Current Completed Modules

- [x] Multi-Agent Architecture Concept
 - [x] Leader Agent Logic
 - [x] Planner Agent
 - [x] Todo Generator
 - [x] Builder Agent System
 - [x] Frontend Builder Agents
 - [x] Backend Builder Agents
 - [x] Database Builder Agents
 - [x] WebContainer Integration
 - [x] Core Code Generation
 - [x] Initial MVP Architecture
-

4. Main Builder Brain Architecture Diagram

```
flowchart TD
    A[User Prompt] --> B[Leader Agent]
    B --> C{Enough Context Available?}
    C -->|No| D[Question Generator Agent]
    D --> E[User Responses]
    E --> F[Planner Agent]
    C -->|Yes| F
    F --> G[Project Planning]
    G --> H[Todo Generator Agent]
    H --> I[Approval Layer]
    I -->|Approved| J[Builder Orchestrator]
    J --> K1[Frontend Builder Agents]
    J --> K2[Backend Builder Agents]
    J --> K3[Database Builder Agents]
    K1 --> L1[Framework-Specific Agents]
    K2 --> L2[API & Logic Agents]
```

```

K3 --> L3[Schema & DB Agents]

L1 --> M[Code Generation Engine]
L2 --> M
L3 --> M

M --> N[Container / WebContainer]

N --> O[Execution & Build Process]

O --> P[File Generation]

P --> Q[Project Summary Generator]
P --> R[Documentation Generator]
P --> S[GitHub Integration]

Q --> T[Final Output]
R --> T
S --> T

T --> U[User]

```

5. Detailed Multi-Agent Architecture

```

flowchart LR
    A[Leader Agent]
    A --> B1[Requirement Analyzer]
    A --> B2[Context Validator]
    A --> B3[Question Decision Engine]
    B1 --> C[Planner Agent]
    C --> D1[Architecture Planner]
    C --> D2[Folder Structure Planner]
    C --> D3[Tech Stack Planner]
    C --> D4[Dependency Planner]
    D1 --> E[Todo Generator]
    E --> F1[Frontend Tasks]
    E --> F2[Backend Tasks]
    E --> F3[Database Tasks]
    F1 --> G1[React Agent]
    F1 --> G2[Angular Agent]
    F1 --> G3[HTML CSS JS Agent]

```

```

F1 --> G4[Next.js Agent]

F2 --> H1[Node Agent]
F2 --> H2[Express Agent]

F3 --> I1[Supabase Agent]

G1 --> J[Builder Agent]
G2 --> J
G3 --> J
G4 --> J

H1 --> J
H2 --> J

I1 --> J

J --> K[Container Executor]

K --> L[Generated Application]

```

6. Memory Architecture Diagram

```

flowchart TD
    A[Conversation Input] --> B[Memory Manager]
    B --> C1[Short-Term Memory]
    B --> C2[Long-Term Memory]
    B --> C3[Project Memory]
    B --> C4[Session Context]
    C1 --> D[Active Agents]
    C2 --> D
    C3 --> D
    C4 --> D
    D --> E[Improved Contextual Responses]

```

7. Document Processing Flow Diagram

```

flowchart TD
    A[Document Upload] --> B[Parser Engine]

```

```

B --> C1[PDF Parser]
B --> C2[DOCX Parser]
B --> C3[Text Extractor]

C1 --> D[Embedding Generator]
C2 --> D
C3 --> D

D --> E[Vector Storage]

E --> F[Context Retrieval]

F --> G[Agents Use Context]

```

8. Container & Sandbox Flow Diagram

```

flowchart TD
    A[Generated Code] --> B[WebContainer]
    B --> C[Install Dependencies]
    C --> D[Run Build]
    D --> E[Test Execution]
    E --> F[Error Detection]
    F --> G{Errors Found?}
    G -->|Yes| H[Fix Agent]
    H --> B
    G -->|No| I[Preview Generated]
    I --> J[Sandbox Integration]

```

9. Document-to-Project Generation Flow

```

flowchart TD
    A[User Uploads Document] --> B[Document Upload Service]

```

```
B --> C[Parser Engine]

C --> D1[PDF Parser]
C --> D2[DOCX Parser]
C --> D3[TXT Parser]

D1 --> E[Content Extraction]
D2 --> E
D3 --> E

E --> F[Requirement Understanding AI]

F --> G1[Frontend Requirement Analysis]
F --> G2[Backend Requirement Analysis]
F --> G3[Database Requirement Analysis]

G1 --> H[Planner Agent]
G2 --> H
G3 --> H

H --> I[Todo Generator]

I --> J[Builder Agents]

J --> K1[Frontend Builder]
J --> K2[Backend Builder]
J --> K3[Database Builder]

K1 --> L[Code Generation Engine]
K2 --> L
K3 --> L

L --> M[WebContainer / Sandbox]

M --> N[Run Application]

N --> O[Generate Preview]

O --> P[Generate Documentation]

P --> Q[Final Project Output]
```

10. Complete Master Todo List

PHASE 1 — MCP + SUPABASE INTEGRATION

MCP Integration

- ☐ Setup MCP server connection
 - ☐ Connect Builder Brain with MCP
 - ☐ Create MCP communication layer
 - ☐ Test MCP request/response flow
 - ☐ Handle MCP failures
 - ☐ Add MCP logging system
-

Supabase Integration

- ☐ Connect Supabase MCP
 - ☐ Setup Supabase authentication
 - ☐ Create database connection layer
 - ☐ Create schema generation logic
 - ☐ Auto table creation system
 - ☐ Auto relationship generation
 - ☐ Database migration handling
 - ☐ Test database CRUD operations
 - ☐ Handle database errors
 - ☐ Add retry mechanisms
-

PHASE 2 — CODE GENERATION IMPROVEMENTS

Frontend Code Generation

- ☐ Improve React code generation
 - ☐ Improve Angular generation
 - ☐ Improve Next.js generation
 - ☐ Tailwind component generation
 - ☐ Reusable component generation
 - ☐ Responsive design handling
 - ☐ Dynamic routing generation
 - ☐ State management generation
 - ☐ Form generation logic
 - ☐ API integration generation
-

Backend Code Generation

- ☐ Express route generation
 - ☐ Controller generation
 - ☐ Middleware generation
 - ☐ Authentication system generation
 - ☐ JWT implementation
 - ☐ Validation system generation
 - ☐ Error handling generation
 - ☐ API documentation generation
-

Database Generation

- ☐ Schema planning logic
 - ☐ Dynamic database models
 - ☐ Relationship generation
 - ☐ Query generation
 - ☐ Database optimization layer
-

PHASE 3 — CONTAINER + PREVIEW SYSTEM

WebContainer Improvements

- ☐ Improve file mounting
 - ☐ Improve dependency installation
 - ☐ Improve project startup flow
 - ☐ Improve build execution
 - ☐ Improve terminal handling
 - ☐ Improve preview generation
 - ☐ Handle preview crashes
 - ☐ Add auto restart system
 - ☐ Add build status tracking
-

Sandbox Integration

- ☐ Setup sandbox environment
 - ☐ Connect sandbox with orchestration
 - ☐ Transfer generated files to sandbox
 - ☐ Execute projects inside sandbox
 - ☐ Add isolated runtime execution
 - ☐ Add process monitoring
 - ☐ Add resource management
 - ☐ Add security restrictions
 - ☐ Test sandbox lifecycle
-

PHASE 4 — ORCHESTRATION IMPROVEMENTS

Workflow Engine

- ☐ Improve agent communication
 - ☐ Add centralized orchestration
 - ☐ Add task dependency handling
 - ☐ Add retry mechanisms
 - ☐ Add fallback agents
 - ☐ Add event-based workflow
 - ☐ Add execution queue
 - ☐ Add task priority system
-

Planning Improvements

- ☐ Better project understanding
 - ☐ Better requirement extraction
 - ☐ Better architecture planning
 - ☐ Better framework selection
 - ☐ Better task decomposition
-

PHASE 5 — MEMORY SYSTEM

Memory Layer

- ☐ Create short-term memory
 - ☐ Create long-term memory
 - ☐ Create session memory
 - ☐ Create project memory
 - ☐ Create agent memory sharing
 - ☐ Create memory retrieval system
 - ☐ Create vector embedding pipeline
 - ☐ Create semantic search layer
-

PHASE 6 — DOCUMENT UNDERSTANDING

Document Upload System

- ☐ File upload UI
- ☐ File validation
- ☐ PDF parser
- ☐ DOCX parser
- ☐ TXT parser

- ☐ Content extraction
 - ☐ Metadata extraction
 - ☐ Embedding generation
 - ☐ Store parsed content
 - ☐ Context retrieval system
-

AI Requirement Understanding

- ☐ Understand uploaded documents
 - ☐ Extract project requirements
 - ☐ Extract frontend requirements
 - ☐ Extract backend requirements
 - ☐ Extract database requirements
 - ☐ Generate project plan from docs
 - ☐ Generate todos from docs
-

PHASE 7 — SKILLS SYSTEM

Frontend Skills

- ☐ UI design skill
 - ☐ Responsive design skill
 - ☐ Animation skill
 - ☐ Accessibility skill
-

Backend Skills

- ☐ API optimization skill
 - ☐ Security skill
 - ☐ Performance optimization skill
-

Database Skills

- ☐ Query optimization skill
 - ☐ Schema optimization skill
-

PHASE 8 — TESTING + DEBUGGING

Debugging System

- ☐ Error analyzer agent

- ☐ Stack trace analyzer
 - ☐ Auto-fix agent
 - ☐ Retry execution system
 - ☐ Bug reporting system
-

Testing System

- ☐ Frontend testing
 - ☐ Backend testing
 - ☐ API testing
 - ☐ Database testing
 - ☐ End-to-end testing
-

PHASE 9 — OUTPUT GENERATION

Documentation

- ☐ README generator
 - ☐ Architecture summary
 - ☐ File explanation system
 - ☐ API docs generator
 - ☐ Setup guide generator
-

GitHub Integration

- ☐ Repository creation
 - ☐ File push automation
 - ☐ Commit generation
 - ☐ Branch handling
 - ☐ Pull request generation
-

PHASE 10 — FUTURE EXPANSION

New Frontend Frameworks

- ☐ Vue.js
 - ☐ Svelte
 - ☐ Nuxt.js
-

New Backend Frameworks

- ☐ Flask
 - ☐ Django
 - ☐ FastAPI
 - ☐ Spring Boot
-

New Databases

- ☐ PostgreSQL
 - ☐ MongoDB
 - ☐ MySQL
 - ☐ Redis
-

Mobile Development

- ☐ React Native
 - ☐ Flutter
 - ☐ Mobile app orchestration
-

11. Future Expansion Vision

Builder Brain is evolving into:

- AI Software Architect
- AI Full-Stack Developer
- AI Planning System
- AI Runtime System
- AI DevOps Assistant
- AI Knowledge Engine
- AI Deployment Platform
- Autonomous Multi-Agent Coding Ecosystem

The long-term goal is to create a complete AI-native software engineering operating system capable of planning, generating, executing, debugging, documenting, and deploying full-stack applications autonomously.